Models Arising from a Flexible Manufacturing Machine, Part I: Minimization of the Number of Tool Switches
Author(s): Christopher S. Tang and Eric V. Denardo
Source: *Operations Research*, Vol. 36, No. 5 (Sep. - Oct., 1988), pp. 767-777
Published by: INFORMS
Stable URL: http://www.jstor.org/stable/171322
Accessed: 08/05/2014 20:46

Your use of the JSTOR archive indicates your acceptance of the Terms & Conditions of Use, available at http://www.jstor.org/page/info/about/policies/terms.jsp

JSTOR is a not-for-profit service that helps scholars, researchers, and students discover, use, and build upon a wide range of content in a trusted digital archive. We use information technology and tools to increase productivity and facilitate new forms of scholarship. For more information about JSTOR, please contact support@jstor.org.

INFORMS is collaborating with JSTOR to digitize, preserve and extend access to *Operations Research*.

http://www.jstor.org

This content downloaded from 169.229.32.137 on Thu, 8 May 2014 20:46:32 PM
All use subject to JSTOR Terms and Conditions

MODELS ARISING FROM A FLEXIBLE MANUFACTURING MACHINE, PART I: MINIMIZATION OF THE NUMBER OF TOOL SWITCHES

CHRISTOPHER S. TANG
University of California, Los Angeles, California

ERIC V. DENARDO
Yale University, New Haven, Connecticut

(Received August 1986; revision received June 1987; accepted October 1987)

This paper considers a job scheduling problem for a flexible manufacturing machine. This problem arises in the metal working industry when numerical controlled (N/C) machines are used to manufacture parts (jobs). Assume that each of N jobs must be processed on a machine that has positions for C tools, but no job requires more than C tools. If the requisite tools are not on the machine, then one or more tool switches must occur before the job can be processed. The problem consists of finding the sequence in which to process the jobs and the tools to place on the machine before each job is processed. This paper considers one performance criterion: minimize the total number of tool switches. This criterion is adequate for situations where the tool switching time is proportional to the number of tool switches. In our model, we assume that the time needed to switch tools is significant relative to the job processing time. A feasible solution is in "equilibrium" if better performance cannot be attained by changing either the job sequence or the tool placement specified by the solution unilaterally. Procedures that find an equilibrium are presented in this paper.

The problem addressed in this paper arises in the metal working industry when numerically controlled (N/C) machines are used to manufacture parts. Each machine performs operations by using the tools placed in its limited capacity tool magazine. Stecke and Brown (1985) presented an extensive description of this kind of flexible manufacturing machine. Stecke (1983) and Stecke and Talbot (1983) considered a machine loading problem in which operations are allocated and tools are assigned to machine groups so that a certain efficiency is attained. They assumed that each machine has an automatic tool interchanging device that can interchange tools in negligible time. However, our industrial experience is somewhat different from theirs. We examined a few types of N/C machines that require fine tuning during tool changes. We have also noticed that the time needed for tuning operations is significant relative to the job processing time. This implies that a lot of machine processing time will be wasted on tuning operations unless the total number of tool changes is minimized. This motivates us to consider a machine loading problem in which the total number of tool changes is minimized.

In this paper, we consider the case in which the tools for all operations are kept in the tool storage area. Each machine has an automatic tool interchanging device that can switch tools between the magazine and the tool storage area. If the requisite tools are not placed on the magazine, then one or more tool switches must occur before the job can be processed. To solve this problem, one selects the sequence in which to process the jobs and the tools to switch on the machine before each job is processed. This paper considers one performance criterion: minimize the number of tool switches. Our companion paper, Tang and Denardo (1988) treats the same problem with a different performance criterion: minimize the number of times at which tools are switched. The former criterion relates to the case in which the time needed to switch tools is significant relative to the job processing time. The latter criterion is appropriate when tool switching is accomplished in parallel. Curiously, these two criteria led to entirely different solution techniques. To our knowledge, the problem considered in this paper is needed for planning prior to production and is studied here for the first time.

In our model, we assume that there is a set of jobs to be processed on a given day, and that this set of jobs is known at the beginning of the day. These jobs are numbered 1 through N. The tools are numbered 1 through M. We assume that each job requires a specific set of tools. This implies that job j has the $M \times 1$ tool requirement vector A_j whose ith entry A_{ji} equals 1 (or 0) if tool i is (is not) required by job j.

Subject classification: Production/scheduling: material handling. Programming: integer, algorithm, heuristic.

0030-364X/88/3605-0767 $01.25
© 1988 Operations Research Society of America

This content downloaded from 169.229.32.137 on Thu, 8 May 2014 20:46:32 PM
All use subject to JSTOR Terms and Conditions

Furthermore, we assume that each tool occupies only one tool slot on the tool magazine. Since the tool magazine has C tool slots and each job requires no more than C tools, $eA_j \leq C$ for each j, where e is a $1 \times M$ vector of 1's.

There are two types of decision variables: the sequence in which to process the jobs, and the set of tools placed on the machine before each job is processed. Let $x_{jn} = 1$ if job j is the nth job in the sequence, and $x_{jn} = 0$ otherwise. The moment in time *after* processing the nth job, but *before* any tools are switched is called *instant n*. Let W_n be the $M \times 1$ vector that describes the tools on the machine at instant n; specifically, the ith element of W_n equals 1 (or 0) if tool i is on (not on) the machine at instant n. A feasible job schedule satisfies these constraints:

$$eW_n \leq C \quad \text{for } n = 1, \ldots, N \tag{1}$$

$$x_{jn}A_j \leq W_n \quad \text{for } j = 1, \ldots, N \quad n = 1, \ldots, N \tag{2}$$

$$\sum_{j=1}^{N} x_{jn} = 1 \quad \text{for } n = 1, \ldots, N \tag{3}$$

$$\sum_{n=1}^{N} x_{jn} = 1 \quad \text{for } j = 1, \ldots, N \tag{4}$$

$$W_n \text{ is a 0–1 vector and } x_{jn} \in \{0, 1\} \quad \text{for all } n, j. \tag{5}$$

Constraint 1 ensures that no more than C tools can be placed on the machine at any instant. Constraint 2 indicates that if job j is the nth job to be processed, then all the tools required by job j must be on the machine at instant n. Constraints 3 and 4 assign exactly one job to exactly one instant. Finally, constraint 5 denotes the integrality requirement for each x_{jn} and W_n.

A *tool switch* occurs when a tool is removed from the machine and a different tool is inserted on the machine. We assume that the time needed for start-up at the beginning of the day and shutdown at the end of the day are fixed. This assumption holds where the start-up and shutdown are part of the standard operating procedures. We also assume that no more time is needed to remove a tool i and insert a tool i' at the same instant than at different instants. Furthermore, we assume that all the operations that are required for each job are performed on the same machine. This implies that there is no tool sharing among different machines. In this case, it is advantageous to insert C tools on the machine at the beginning (instant 0), switch tools if necessary at each instant n (where $n = 1, \ldots, N - 1$), and remove C tools from the machine at the end of the job sequence (instant N). Consequently, there are exactly C tools on the machine at each instant, and hence, constraint 1 can be tightened as follows.

$$eW_n = C \quad \text{for } n = 1, \ldots, N. \tag{6}$$

To count the tool switches that occur at instant n, we introduce the $M \times 1$ vector P_n and the constraints

$$P_n \geq W_{n+1} - W_n \quad \text{for } n = 1, \ldots, N-1, \tag{7}$$

$$P_n \geq 0 \quad \text{for } n = 1, \ldots, N-1. \tag{8}$$

Note that no tool switches occur at instant N, as discussed previously. The problem of minimizing the total number of tool switches is expressed as follows.

Program I

$$\text{Minimize} \sum_{n=1}^{N-1} eP_n$$

subject to (2)–(8).

Section 1 of this paper describes the rather disappointing experience that we have had applying standard integer programming codes to this problem. Section 2 observes how to find the best tool sequence (the W_n's) with a fixed job sequence (fixed x_{jn}'s). Section 3 describes a heuristic procedure that finds a solution that is locally optimal in the sense that improvements cannot be found by varying either the x_{jn}'s or the W_n's. Section 4 describes our computational experience with this procedure, and Section 5 concludes this paper with a summary.

1. Computational Experience

Let us consider a simple example first. In this example, there are 10 jobs ($N = 10$) to be processed on a machine that can hold 4 tools ($C = 4$). These ten jobs require a total of 9 different tools ($M = 9$). The tool requirement vectors are given in Table I.

This problem was used as our initial investigation of the computational effort required for solving Program I. In this case, the corresponding mixed integer program has 190 0–1 variables and 90 continuous variables. Our first step was to see how MPSX/MIP (the Mixed Integer Programming code for IBM computers) fared on these data. We ran it for a total of 12 minutes of CPU time on an IBM 370. In the first 8 minutes, it generated 2 feasible solutions; the better one entailed 11 tool switches. No better solutions were found in the remaining 4 minutes of CPU time. Our next step was to see how PIPX (the pure 0–1 programming code developed by Crowder, Johnson and

This content downloaded from 169.229.32.137 on Thu, 8 May 2014 20:46:32 PM
All use subject to JSTOR Terms and Conditions

Table I
The Tool Requirement Vectors

	Job									
Tool	1	2	3	4	5	6	7	8	9	10
1	1	1	0	0	0	0	1	0	0	1
2	0	0	1	0	0	0	0	0	0	1
3	0	1	0	0	0	1	0	1	0	0
4	1	0	0	0	0	0	0	0	0	1
5	0	1	0	0	0	0	1	1	1	0
6	0	0	1	0	1	0	0	0	0	0
7	0	0	1	1	0	0	1	0	1	0
8	1	0	1	0	0	0	0	1	0	0
9	1	0	0	0	0	0	1	0	0	0

Padberg 1983) fared on these data. We ran the same problem for a total of 10 minutes of CPU time on an IBM 370. It generated several feasible solutions; the best entailed 8 tool switches. This is slow for a mixed integer program of this size. Similar results were observed when constraint 2 was tightened in the following way.

$$\sum_{j=1}^{N} x_{jn}A_j \leq W_n \quad \text{for } n = 1, \ldots, N. \tag{9}$$

It can be easily checked that any solution that satisfies (3)–(9) will also satisfy (2)–(8), but not conversely. Hence, we can conclude that constraint 9 is tighter than constraint 2.

After these disappointing starts, we considered a subproblem of the job scheduling problem that is discussed in the next section.

2. The Tool Replacement Problem

The *tool replacement problem* is the special case of our model in which the job sequence is fixed (i.e., x_{jn}'s are fixed and satisfy constraints 3–5), and the tooling decisions (i.e., W_n's) are the only decisions to be considered. The objective of this problem is to determine the set of tools to be placed on the machine at each instant so that the total number of tool switches is minimized. A *Keep Tool Needed Soonest* (KTNS) policy has the following properties:

1. At any instant, no tool is inserted unless it is required by the next job.
2. If a tool must be inserted, the tools that are kept (not removed) are those needed the soonest.

We shall see that any KTNS policy is optimal for the tool replacement problem. Precise specification of the KTNS policy entails two definitions. The set $T(i, n)$ is the set of all instants at or after instant n at which tool i is needed, and the integer $L(i, n)$ is the first instant at or after instant n at which tool i is needed. More formally, we let

$$T(i, n) = \{m - 1 : \exists j, m > n, A_{ji} = 1 \text{ and } x_{jm} = 1\} \cup \{N\},$$

$$L(i, n) = \min\{m : m \in T(i, n)\}.$$

Note that for a complete definition of $L(i, n)$, we set $L(i, n)$ equal to N if tool i is not needed after instant n. The KTNS policy is described by the following procedure. We let J be the vector whose ith entry J_i equals 1 (or 0) if tool i is (is not) on the machine at a given instant n.

KTNS Procedure

Step 1. Set $J_i = 1$ for C values of i having minimal values of $L(i, 0)$. Break the ties arbitrarily. Set $J_i = 0$ for the remaining $M - C$ values of i. Set $n = 1$.

Step 2. Set $W_n = J$. Stop if $n = N$.

Step 3. If each i having $L(i, n) = n$ also has $J_i = 1$, set $n = n + 1$ and go to Step 2.

Step 4. Pick i having $L(i, n) = n$ and $J_i = 0$. Set $J_i = 1$.

Step 5. Set $J_k = 0$ for a k that maximizes $L(p, n)$ over $\{p: J_p = 1\}$. Go to Step 3.

It can be easily seen that Steps 2 and 3 assure that W_n consists of all the tools required by the nth job in the sequence. Also, note that the tools to be inserted (property 1 of the KTNS policy) are considered in Steps 1 and 4. Furthermore, the tools to be kept (property 2 of the KTNS policy) are determined by the tools to be removed from the machine, which are considered in Step 5.

Theorem 1. *Each KTNS policy minimizes the total number of tool switches for the tool replacement problem.*

The proof of Theorem 1 is given in the Appendix. It can be easily seen that the amount of work required by the KTNS policy is on the order $O(MN)$. Based on the simplicity of the KTNS policy, Theorem 1 suggests that the tool replacement problem is a quickly solved subproblem. The job scheduling problem is to find the optimal job schedule based on the assumption that the tool replacement problem is solved optimally for every job schedule. Therefore, it is clear that the optimal job schedule can be determined by solving the tool replacement problem for each job schedule. However, this approach is undesirable because the total number of job sequences is $N!$, where N is the

This content downloaded from 169.229.32.137 on Thu, 8 May 2014 20:46:32 PM
All use subject to JSTOR Terms and Conditions

total number of jobs to be processed on the machine. This observation is not surprising because the job scheduling problem is a complex combinatorial problem.

To examine the complexity of the job scheduling problem, let us consider a complete graph with N nodes, each of which represents a job. The length of each arc between any two nodes (jobs) represents the number of tool switches incurred when this pair of jobs is processed consecutively. Note that the number of tool switches between any two jobs depends on the tooling decisions. Then every job sequence corresponds to a Hamiltonian path on the graph, that is, a path that encounters each node exactly once. Moreover, the length of each Hamiltonian path represents the total number of tool switches entailed by the corresponding job sequence. Therefore, finding an optimal job sequence for the job scheduling problem amounts to finding the shortest Hamiltonian path on a complete graph with variable arc lengths. It is not difficult to see that finding the shortest Hamiltonian path on a complete graph is equivalent to solving the Traveling Salesman Problem (TSP). TSP is a well known NP-complete problem. Hence, we can conclude that the job scheduling problem is also NP-complete. This motivates us to develop a heuristic approach for finding a job scheduling that is good, but not necessarily optimal, for the job scheduling problem.

Before we present the heuristic for the job scheduling problem, let us consider the situation as follows. For any job i, let $F(i)$ be the set of jobs that requires only a subset of the tools required by job i. More formally, we let $F(i) = \{j: j \neq i, A_j \leq A_i\}$. In this case, no tool switches are required if all the jobs in $F(i)$ (if any) are scheduled in any order immediately after job i. Therefore, it must be optimal to schedule the set of jobs $F(i)$ immediately after job i. Let us consider the tool requirement vectors given in Table I. It can be seen that $A_4 \leq A_3$ and $A_5 \leq A_3$, hence $F(3) = \{4, 5\}$. In this case, no tool switches are required for job sequence (3, 4, 5) or (3, 5, 4). Based on this observation, we reduce the original set of ten jobs to these six: $\{1, 2, 3, 7, 8, 10\}$.

3. A Heuristic for Job Scheduling

In this section, we present our heuristic approach to the job scheduling problem. This approach consists of three major steps. The first step generates a good job schedule. The second step employs the KTNS policy to determine the total number of tool switches required by the job schedule. The third step finds a better job schedule.

The first step consists of finding a short Hamiltonian path on the aforementioned complete graph except that the arc length between each pair of nodes (jobs) is approximated by the fewest number of tool switches needed by that pair of jobs when processed sequentially. More formally, let LB(i, j) denote the fewest possible number of tool switches needed if job j follows job i in the job sequence. The number of tools not required by job i equals $C - eA_i$. The number of tools needed by job j, but not job i, equals $(eA_j - A_j^T A_i)$. Hence, by holding a tool for job j in each position not required by job i, we see that

$$\begin{aligned} \mathrm{LB}(i,j) &= [(eA_j - A_j^T A_i) - (C - eA_i)]^+ \\ &= [e(A_i + A_j) - (C + A_j^T A_i)]^+. \end{aligned}$$

Note that LB(i, j) = LB(j, i). The fewest number of tool switches among those six remaining jobs (i.e., LB(i, j)) are reported in Table II.

Note that LB(i, j) ignores all the tooling decisions before job i and the tooling decisions after job j in the job sequence. This implies that the fewest number of tool switches LB(i, j) is determined in a myopic sense, and it may not be attainable. It follows that the length of each Hamiltonian path is a lower bound on the number of tool switches needed to execute the corresponding job sequence. Consequently, the length of a shortest Hamiltonian path is a lower bound on the number of tool switches incurred by the optimal job schedule.

In our example, there are six remaining jobs to be considered. Also, it can be easily checked that the six shortest Hamiltonian paths are on the complete graph, each has length 6 (i.e., the sum of all the arc lengths of each path). Each path corresponds to a job sequence. The total number of tool switches required by each job sequence is determined by the KTNS policy and is reported in Table III.

Five of the six job sequences entail seven tool

Table II
LB(i, j)

	j					
i	1	2	3	7	8	10
1	—	2	3	2	2	1
2		—	3	1	0	1
3			—	3	2	2
7				—	2	2
8					—	2
10						—

This content downloaded from 169.229.32.137 on Thu, 8 May 2014 20:46:32 PM
All use subject to JSTOR Terms and Conditions

Table III
Total Tool Switches Needed by Each Job Sequence

Schedule	Job Sequence	Number of Tool Switches
a	3, 10, 1, 7, 2, 8	7
b	1, 10, 3, 8, 2, 7	7
c	3, 8, 2, 10, 1, 7	8
d	1, 10, 7, 2, 8, 3	7
e	3, 10, 1, 8, 2, 7	7
f	3, 8, 2, 7, 1, 10	7

switches. These five job sequences must be optimal because the Hamiltonian path corresponding to any other job sequence has, at least, length seven, and hence, entails at least seven tool switches. Therefore, the optimal job schedules for this example are a, b, d, e, and f.

The second step employs the KTNS policy to determine the tooling decision for any given job schedule. The third step consists of finding a job schedule that may incur a fewer number of tool switches than the current best job schedule. This can be achieved by selecting the best job sequence out of all those job sequences that can be performed by the same sequence of tools that is used to process the current best job schedule. More formally, let J denote the current best job schedule. Let $W^J(n)$ denote the set of tools on the machine at instant n and TS(J) denote the number of tool switches entailed by the job schedule J that are determined by the KTNS policy. We call a job sequence Q a *perturbed* job sequence of J if Q can be performed by the same sequence of tools that is used to process the job sequence J; i.e., Q is a set of x_{jn}'s that satisfies constraints 2–5, for fixed $W^J(n)$'s. Note that $W^J(n)$ is a feasible tooling decision for the job sequence Q. This implies that the optimal tooling decision for the job sequence Q will entail no more than TS(J) tool switches; i.e., TS(Q) $\leq$ TS(J). This observation suggests that it is possible to find a better job sequence by evaluating all the perturbed job sequences of J, where J is the current best job sequence. Furthermore, explicit enumeration is used in our heuristic for finding a perturbed job sequence of J because the total number of perturbed job sequences of any job sequence is rather small in practice.

Combining these three major steps, we develop a procedure that is called the Greedy Perturbation (GP) procedure. This procedure can be described as follows.

GP Procedure

Step 1. [Initiation] (Greedy Procedure). (a) Assign the length LB(i, j) to each arc (i, j) on the complete graph. Pick the shortest arc on the graph and put it on the list $\mathscr{L}$. Remove the arc from the graph. (b) If $\mathscr{L}$ consists of $N - 1$ arcs, then let J be the corresponding Hamiltonian path and go to Step 2; otherwise, continue. (c) Pick the shortest arc on the remaining graph. (d) Check the arc picked in c with all the arcs on the list $\mathscr{L}$. If that arc forms a tree (a node with degree 3) or a cycle with some of the arcs on the list $\mathscr{L}$, then remove that arc from the graph and go to c; otherwise, add that arc to the list $\mathscr{L}$ and go to b.

Step 2. [Evaluation] (KTNS Procedure). Use the KTNS procedure to determine $W^J(n)$ and TS(J).

Step 3. [Enumeration] (Perturbation Procedure). (a) Set $\mathscr{M} = J$. (b) Find a job sequence Q that is a perturbed job sequence of J and is not on the list $\mathscr{M}$. Stop if none is found, else add Q to the list $\mathscr{M}$ and continue. (c) Use the KTNS policy to determine $W^Q(n)$ and TS(Q). If TS(Q) < TS(J), then set $J = Q$ and set TS(J) = TS(Q). Go to b.

Observe that the Greedy Procedure (Step 1) employs a greedy heuristic to generate a job sequence by sequentially selecting pairs of jobs. The selection criterion is based on the minimum arc length LB(i, j) on the graph, where LB(i, j) is the fewest tool switches between jobs i and j.

It can be easily checked that the Greedy Perturbation (GP) procedure stops and finds a job sequence J that has the following properties:

1. For a fixed job schedule J, W^J is an optimal tooling decision for the job schedule J.
2. For a fixed tooling decision W^J, there is no perturbed job sequence of J that entails less than TS(J) tool switches.

We call any job sequence J and its tooling decision W^J, that is, (J, W^J), an *equilibrium* to the job scheduling problem if it satisfies Properties 1 and 2. Hence we can conclude that an equilibrium can be found by the GP procedure described above.

4. Computational Results

In order to investigate the effectiveness of the GP procedure for the job scheduling problem with one machine, we generate random problems with the following characteristics.

1. We consider the following four cases. There are N = 10, 20, 30 or 40 jobs to be processed on the machine that needs M = 10, 15, 25 or 30 tools altogether, respectively. Correspondingly, we set

This content downloaded from 169.229.32.137 on Thu, 8 May 2014 20:46:32 PM
All use subject to JSTOR Terms and Conditions

the tool capacity on the machine to $C = 4$, 8, 10 and 15.

2. In each of the first three cases, we consider five sets of data that are based on the random tool requirements A_j for each job j. We consider that the number of tools required by each job is uniformly distributed over $[1, C]$. We consider two sets of data in the fourth case. For each set of data, we make sure that the set of tools required by each job is not a subset of the set of tools required by any other job. More formally, there does not exist jobs j and k so that $A_j \leq A_k$.
3. For each data set in the first two cases, we employ the KTNS policy to determine the tool switches entailed by each of 100 random job sequences that are generated from random permutation of jobs. In a similar fashion, we employ the KTNS policy to determine the tool switches entailed by each of 200 random job sequences with respect to the data sets in the last two cases.
4. For each data set in each case, we ran the GP procedure for, at the most, 2 minutes of CPU time.

Tables IV–VII summarize our computational experience on an IBM 370, where all the procedures are written in APL. The following notation is used for each table.

Min = The number of tool switches entailed by the best of all random starting job sequences.
Max = The number of tool switches entailed by the worst of all random starting job sequences.

Table IV
GP Procedure Results for Case 1:
$N = 10, M = 10, C = 4$

Data Set	Min	Max	Mean	Sigma	TS	Term
1	6	13	9.45	1.22	6	Y
2	11	14	12.66	0.20	9	Y
3	11	14	12.43	0.22	9	Y
4	10	15	13.17	0.24	11	Y
5	12	21	15.75	1.45	9	Y

Table V
GP Procedure Rsesults for Case 2:
$N = 20, M = 15, C = 8$

Data Set	Min	Max	Mean	Sigma	TS	Term
1	38	54	45.81	3.36	28	Y
2	27	41	32.84	2.62	21	Y
3	36	52	44.10	3.26	29	Y
4	33	47	40.28	2.72	29	Y
5	23	33	27.18	1.97	17	Y

Table VI
GP Procedure Results for Case 3:
$N = 30, M = 25, C = 10$

Data Set	Min	Max	Mean	Sigma	TS	Term
1	72	89	81.53	3.59	54	N
2	83	103	92.91	4.26	75	N
3	70	94	81.17	4.07	60	N
4	75	95	87.13	4.10	69	N
5	84	107	95.21	4.92	72	N

Table VII
GP Procedure Results for Case 4:
$N = 40, M = 30, C = 15$

Data Set	Min	Max	Mean	Sigma	TS	Term
1	111	135	123.9	5.07	107	N
2	118	145	132.9	5.31	110	N

Mean = The average number of tool switches entailed by all the random starting job sequences.
Sigma = The standard deviation of the tool switches entailed by all the random starting job sequences.
TS = The number of tool switches entailed by the job sequence found by the GP procedure.
Term = (Y or N.) Indicate whether the GP procedure terminates within 2 minutes of CPU time.

Based on the results shown in Tables IV–VII, it seems that the GP procedure is efficient when $N = 10$, 20, and 30. It can be easily checked that the number of tool switches incurred by the job sequence found by the GP procedure is approximately 5.5 standard deviations below the mean; i.e., TS $\approx$ Mean $-$ 5.5 Sigma. However, it seems that the GP procedure is somewhat less efficient when $N = 40$. This phenonmenon may be caused by the fact that 2 minutes of CPU time is not sufficient for the GP procedure to generate a good solution when N is large.

5. Conclusion

In this paper, we address a job scheduling problem that arises in the automated manufacturing environment. We make a few assumptions that may have limited the applicability of the model. The assumption that concerns the unique set of tool requirements for each job may reduce the flexibility of the machine or tool operations. The assumption that concerns the full loading of the magazine at each instant (i.e., the magazine holds exactly C tools at each instant) may limit the possibility of tool sharing among machine

This content downloaded from 169.229.32.137 on Thu, 8 May 2014 20:46:32 PM
All use subject to JSTOR Terms and Conditions

centers. However, both of these assumptions appear essential for our development; without them, the solution set for the tooling decision becomes extremely large and difficult to characterize. Consequently, the problem will become too hard to analyze. In any event, we hope that the approach addressed in this paper will provide some insights for generating solution techniques to solve some general planning problems that arise in flexible manufacturing.

To summarize, we have presented a heuristic for finding an equilibrium (local optimum) to the job scheduling problem. We first constructed the KTNS policy to determine optimal tool replacements on one machine for any given job schedule. Combining this simple KTNS policy with the Greedy and the Perturbed procedures, we developed the GP procedure to find an equilibrium to the job scheduling problem. Based on our computational results, it occurred to us that the GP procedure is rather efficient in generating good solutions to the job scheduling problem.

Appendix. Proof of Theorem 1

For a fixed job schedule $J = (j_1, j_2, \ldots, j_N)$, let us renumber the jobs so that job j_k is the kth job in the sequence. In this case, instant n is the moment in time just before any tools are switched, but after processing the nth job in the sequence. At instant n, let us recall that $L(i, n)$, as defined in Section 1, is the first instant at or after instant n at which tool i is needed. Moreover, we let $W(n)$ be the set of tools on the machine at instant n. Let us define the following terms that will be used in our discussion:

- The *Kept Set* $\mathrm{K}(n)$, where $\mathrm{K}(n) = W(n) \cap W(n+1)$ for $n = 1, \ldots, N-1$.
- The *Removed Set* $\mathrm{R}(n)$, where $\mathrm{R}(n) = W(n) \setminus W(n+1)$ for $n = 1, \ldots, N-1$.
- The *Inserted Set* $\mathrm{I}(n)$, where $\mathrm{I}(n) = W(n+1) \setminus W(n)$ for $n = 1, \ldots, N-1$.

It can be easily seen that the kept set $\mathrm{K}(n)$ represents the set of tools that remains on the machine at instant n. Similarly, $\mathrm{R}(n)$ and $\mathrm{I}(n)$ represent the set of tools that are removed and inserted at instant n, respectively.

A tool replacement policy g is feasible if and only if all the tool requirements are satisfied at all instants; i.e., the set $\{i: A_{ni} = 1\} \subseteq W^g(n)$ for $n = 1, \ldots, N$, where $W^g(n)$ is the set of tools on the machine at instant n under policy g. Moreover, let $\mathscr{G}$ denote the set of all feasible tool replacement policies for the given job schedule J.

Early Insertions and Reversals

For any tool replacement policy $g \in \mathscr{G}$, we said that g commits an *early insertion* at instant n if there is a tool s that is inserted at instant n, but is not required by job $n + 1$. More formally, let $\mathrm{EI}(g, n)$ denote the total number of early insertions that policy g commits at instant n, where

$$\mathrm{EI}(g, n) = |\mathrm{TI}(g, n)|, \quad \text{and}$$

$$\mathrm{TI}(g, n) = \{i: i \in I(n), L(i, n) > n\}.$$

In a similar fashion, we said that g commits a *reversal* at instant n if there is a tool t that is removed at instant n, and a tool s is kept at instant n such that $L(t, n) < L(s, n)$. More formally, let $\mathrm{ER}(g, n)$ denote the total number of reversals committed by policy g at instant n, where

$$\mathrm{ER}(g, n) = |\mathrm{TR}(g, n)|, \quad \text{with}$$

$$\mathrm{TR}(g, n) = \{s \in \mathrm{K}(n): \exists t \in \mathrm{R}(n), \; L(t, n) < L(s, n)\}.$$

Throughout this section, we specify a tool t that is removed but is needed the soonest among all the tools in the removed set at instant n; i.e., $L(t, n) \leqslant L(u, n)$ for any $u \in R^g(n)$. Also, we partition the kept set into two disjoint sets, namely, $\mathrm{NR}(g, n)$ and $\mathrm{TR}(g, n)$, where $\mathrm{NR}(g, n) = \{v \in K^g(n): L(v, n) \leqslant L(t, n)\}$. This implies that for each $j \in \mathrm{NR}(g, n)$, $L(j, n) \leqslant L(t, n) \leqslant L(u, n)$ for all $u \in R^g(n)$. Finally, we let $f(g)$ be the first instant at which g commits at least one reversal or early insertion or both. In this case, $f(g)$ can take on the value between 0 through $N - 1$. Also, we set $f(g) = N$ if g commits no reversals and no early insertions.

On Optimal Tool Replacement Policies

Lemma 1. *There exists an optimal tool replacement policy with no reversals and no early insertions.*

Proof. Suppose that the hypothesis is false, then any optimal policy commits either at least one reversal or early insertion. In that case, $f(g) < N$ for all $g \in \mathscr{G}^*$, where $\mathscr{G}^*$ denotes the set of all optimal policies. Therefore, we can always choose a policy $g^* \in \mathscr{G}^*$ such that g^* minimizes $[-f(g), \mathrm{ER}(g, f(g)), \mathrm{EI}(g, f(g))]$ *lexicographically* for all $g \in \mathscr{G}^*$. Now let us consider the following cases that arise at instant $f(g^*)$. For notational convenience, we set $n = f(g^*)$.

Case 1. $\mathrm{ER}(g^*, n) > 0$; i.e., g^* commits at least one reversal at instant n.

This content downloaded from 169.229.32.137 on Thu, 8 May 2014 20:46:32 PM
All use subject to JSTOR Terms and Conditions

In this case, there must exist a tool t in the removed set and a tool s in the kept set at instant n, such that $L(s, n) > L(t, n)$. Let p be the next instant after n in which tool t is inserted and let q be the next instant in which tool s is removed. Note that $n < q$ and $n < p \leqslant L(t, n)$, as tool t must be inserted on the machine no later than $L(t, n)$.

Based on this setting, let us consider the following scenarios.

Scenario 1. ($q < p$)

Let g' be the variant of the optimal policy g^* in which tool s is removed at instant n and tool t is removed at instant q, but no other changes are made. Note that g' is a feasible policy, and g' commits no reversals, and no early insertions before instant n. Figure 1 depicts policy g', which is the variation of policy g^*.

It can be easily seen that the total number of tool switches incurred by policy g' is equal to the total number of tool switches incurred by policy g^*. It follows that policy g' is also an optimal policy.

Let us first consider the removed set $R^{g'}(n)$, where $R^{g'}(n) = R^{g^*}(n) \cup \{s\}\backslash\{t\}$. It follows from the facts that $L(t, n) < L(s, n)$ and $L(t, n) \leqslant L(u, n)$ for any $u \in R^{g^*}(n)$, we can conclude that

$$L(t, n) \leqslant L(u, n) \quad \text{for any } u \in R^{g'}(n). \tag{A.1}$$

Based on the definition of the set $\mathrm{NR}(g^*, n)$, we have

$$L(j, n) \leqslant L(t, n) \quad \text{for any } j \in \mathrm{NR}(g^*, n). \tag{A.2}$$

Combining (A.1) and (A.2) with the definition of $\mathrm{NR}(g', n)$, we can check that $\mathrm{NR}(g^*, n) \cup \{t\} \subseteq \mathrm{NR}(g', n)$. This implies that

$$\mathrm{NR}(g^*, n) \subset \mathrm{NR}(g', n). \tag{A.3}$$

Now, let us consider the kept set of policy g' at instant n, where

$$K^{g'}(n) = \mathrm{NR}(g^*, n) \cup \{t\} \cup [\mathrm{TR}(g^*, n)\backslash\{s\}]. \tag{A.4}$$

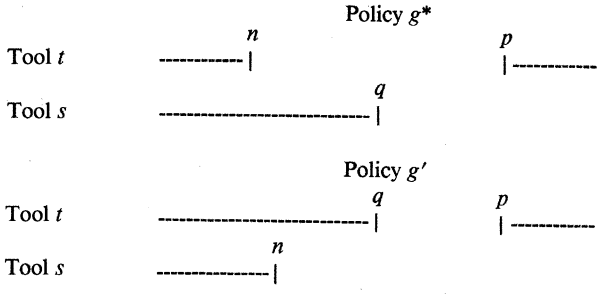

Figure 1. Policy g' for case 1, scenario 1 ($q < p$).

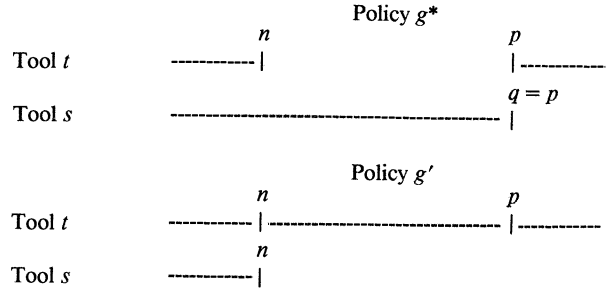

Figure 2. Policy g^* for case 1, scenario 2 ($p = q$).

This implies that

$$|K^{g'}(n)| = |K^{g^*}(n)|. \tag{A.5}$$

Combining (A.3) and (A.5) with the definition of $\mathrm{TR}(g', n)$, we can conclude that $|\mathrm{TR}(g^*, n)| > |\mathrm{TR}(g', n)|$. It follows that $\mathrm{ER}(g^*, n) > ER(g', n)$, and hence, the number of reversals committed by policy g' is straightly less than the number of reversals committed by g^*. This contradicts the assumption that g^* is the policy in $\mathscr{G}^*$ that minimizes $[-f(g), \mathrm{ER}(g, n), \mathrm{EI}(g, n)]$ lexicographically, and hence, this scenario cannot happen.

Scenario 2. ($p = q$)

Let g' to be the variant of the optimal policy g^* in which tool s is removed at instant n and t is kept on the machine from instant n through p, but no other changes are made. Figure 2 depicts the variations of policy g^* as described.

Note that g' is a feasible policy and g' commits no reversals and no early insertions before instant n. The figure indicates that the total number of tool switches incurred by policy g' is reduced by 1 as opposed to policy g^*. This contradicts the assumption that g^* is an optimal policy, and hence, this scenario cannot happen.

Scenario 3. ($q > p$)

Let g' to be the variant of the optimal policy g^* in which tool s is removed at instant n and reinserted at instant p while tool t is kept from instant n through p, but no other changes are made. Figure 3 depicts the variations of policy g^*.

By using the same argument presented in Scenario 1, we can conclude that g' is an optimal policy with a reduced number of reversals at instant n, and hence, this scenario cannot happen.

Summarizing all these scenarios, we conclude that Case 1 cannot happen. Now, let us consider Case 2, where policy g^* commits no reversals but some

This content downloaded from 169.229.32.137 on Thu, 8 May 2014 20:46:32 PM
All use subject to JSTOR Terms and Conditions

early insertions at instant n; i.e., $\mathrm{ER}(g^*, n) = 0$ and $\mathrm{EI}(g^*, n) > 0$.

Case 2. ($\mathrm{ER}(g^*, n) = 0$ and $\mathrm{EI}(g^*, n) > 0$); i.e., g^* commits no reversals but some early insertions at instant n.

In this case, there must exist a tool s that is inserted at instant n, but is not required by job $n + 1$. Let p be the next instant after instant n in which tool t is inserted, where $p > n$.

Based on this setting, let us consider the following scenarios.

Scenario 1. ($p > n + 1$)

Let g' to be the variant of the optimal policy g^* in which tool t is kept at instant n and is removed at instant $n + 1$; also, tool s is inserted at instant $n + 1$, but no other changes are made.

Figure 4 depicts the variations of policy g^* as described. Note that g' is a feasible policy and g' commits no reversals and no early insertions before instant n. Also, g' is an optimal policy as no extra tool switches are incurred in the variation of policy g^*. Furthermore, policy g' commits no extra reversals at instant n as $L(t, n) \leqslant L(u, n)$ for all $u \in R^{g^*}(n)$. It follows that g' commits no reversals at instant n as g^* commits no reversals at instant n. On the other hand, it is clear that policy g' reduces the number of early insertions at instant n. This contradicts that assumption g^* is the policy in $\mathscr{G}^*$ that minimizes $[-f(g), \mathrm{ER}(g, n), \mathrm{EI}(g, n)]$ lexicographically, and hence, this scenario cannot happen.

Scenario 2. ($p = n + 1$)

Let g' to be the variant of the optimal policy g^* in which tool t is kept from instant n through $n + 1$ and tool s is inserted at instant $n + 1$, but no other changes are made. Figure 5 depicts the variations of the policy g^*. Note that g' is a feasible policy and the total

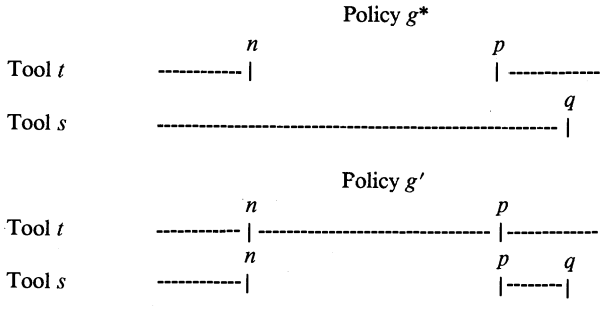

Figure 3. Policy g^* for case 1, scenario 3 ($q > p$).

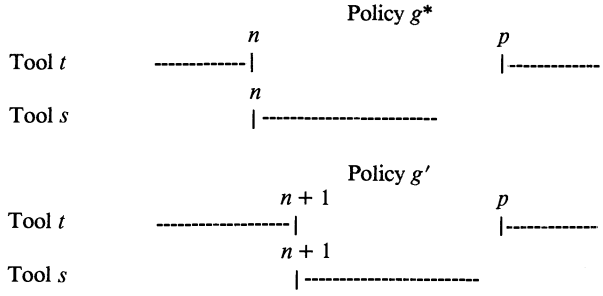

Figure 4. Policy g^* for case 2, scenario 1 ($p > n + 1$).

number of tool switches incurred by policy g' is reduced by 1 as opposed to policy g^*. This contradicts the assumption that g^* is an optimal policy, and hence, this scenario cannot happen.

Summarizing these two scenarios, we conclude that Case 2 cannot happen, and hence, the proof is completed.

It can be easily checked that any KTNS policy is a tool replacement policy that commits no early insertions, no reversals, and vice versa. This fact is now stated in Lemma 2.

Lemma 2. *Any feasible tool replacement policy that commits no early insertions and no reversals is a KTNS policy. Conversely, any KTNS policy commits no early insertions and no reversals.*

We have shown that there exists a KTNS policy that is optimal. In order to show that any KTNS policy is optimal, it is sufficient to show that the total number of tool switches entailed by any two different KTNS policies are equal.

Let us first consider that the number of tool switches occurs at instant n for any policy is equal to the number of tools inserted or removed at instant n; i.e., $|\mathrm{R}(n)| = |\mathrm{I}(n)|$. Also, note that $C - |\mathrm{K}(n)| = |\mathrm{R}(n)| = |\mathrm{I}(n)|$. As each KTNS policy g commits no early insertion at instant n, hence the inserted set consists of tools that are needed at instant $n + 1$, but they are not on the machine at instant n. More formally

$$I^g(n) = \{i\colon L(i, n) = n\} \backslash W^g(n). \tag{A.6}$$

Let us consider the following terms that will be needed in our discussion.

(1) For any given vector v of length m, we said that v is an *increasing vector* if $v_j \geqslant v_i$ for all $m > j > i > 1$.

The notion of increasing vector arises from arranging $L(i, n)$'s in ascending order for all the tools in

This content downloaded from 169.229.32.137 on Thu, 8 May 2014 20:46:32 PM
All use subject to JSTOR Terms and Conditions

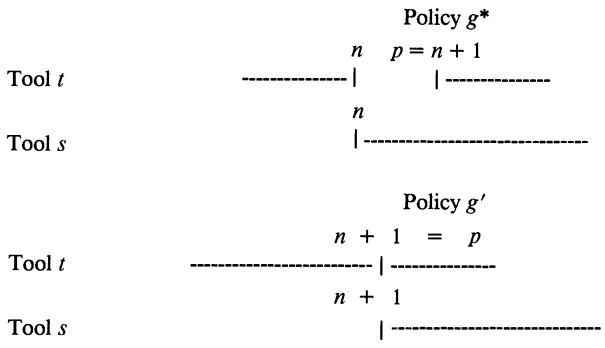

Figure 5. Policy g^* for case 2, scenario 2 ($p = n + 1$).

a set at instant n. Let $A(g, n)$ be the increasing vector that corresponds to the set of tools $W^g(n)$ at instant n.

(2) For any two different policies g and g' in $\mathscr{G}$, $W^g(n)$ is *similar* to $W^{g'}(n)$ at instant n if and only if $A(g, n) = A(g', n)$.

For any two KTNS policies g and g', if $A(g, n) = A(g', n)$ at instant n, then

$$|\{i: L(i, n) = n\} \cap W^g(n)| = |\{i: L(i, n) = n\} \cap W^{g'}(n)|.$$

Also, as g and g' are KTNS policies, hence they commit no early insertions. It follows from (6) that $|I^g(n)| = |I^{g'}(n)|$, and hence, policies g and g' commit the same number of tool switches at instant n. This suggests the following:

$$\text{if } W^g(n) \text{ is similar to } W^{g'}(n) \text{ then } |I^g(n)| = |I^{g'}(n)|. \tag{7}$$

Lemma 3. *The total number of tool switches entailed by any two different KTNS policies g and g' are equal.*

Proof. It follows from (7) that it is sufficient to show that $W^g(n)$ is similar to $W^{g'}(n)$ at each instant n. This can be shown by using mathematical induction on instant n.

Based on Step 1 in the KTNS Procedure, as listed in Section 3, for any KTNS policies g and g', $W^g(1)$ is similar to $W^{g'}(1)$. Now, let us assume that $W^g(n)$ is similar to $W^{g'}(n)$ for $n = 1, \ldots, k$, and hence, $A(g, k) = A(g', k)$. It follows from the fact that g and g' commit no early insertions and the fact that $A(g, k) = A(g', k)$, that $|I^g(k)| = |I^{g'}(k)|$ and $|K^g(k)| = |K^{g'}(k)|$. Combining this with the fact that g and g' commit no reversals, we can conclude that the increasing vectors correspond to the kept sets of policies g and g' are equal at instant k. In that case, the increasing vectors correspond to the sets $K^g(k)\backslash\{i: L(i, k) = k\}$ and the set $K^{g'}(k)\backslash\{i: L(i, k) = k\}$ are equal at instant k. Based on the definition of $L(i, n)$, for any tool i such that $L(i, k) > k$, $L(i, k + 1) = L(i, k)$. In that case, the increasing vectors correspond to the set $K^g(k)\backslash\{i: L(i, k) = k\}$ and the sets $K^{g'}(k)\backslash\{i: L(i, k) = k\}$ are equal at instant $k + 1$. Now, let us consider the set $W^g(k + 1)$ and the set $W^{g'}(k + 1)$, where

$$W^g(k+1) = \{i: L(i, k) = k\} \cup [K^g(k)\backslash\{i: L(i, k) = k\}]$$

and

$$W^{g'}(k+1) = \{i: L(i, k) = k\} \cup [K^{g'}(k)\backslash\{i: L(i, k) = k\}].$$

It then follows that the increasing vectors that correspond to the sets $W^g(k + 1)$ and $W^{g'}(k + 1)$ are equal. Therefore, we conclude that $W^g(n)$ is similar to $W^{g'}(n)$ at all instants, and hence, the proof is completed.

Theorem 2. *Every KTNS policy is an optimal tool replacement policy.*

Proof. In Lemma 3, we proved that the total number of tool switches for all KTNS policies are the same. As shown in Lemma 2, KTNS policy commits no early insertions and no reversals. It then follows from Lemma 1, and we conclude that any KTNS policy is optimal.

Acknowledgment

The authors would like to acknowledge the associate editor and two anonymous referees for their valuable comments on an earlier version of this paper. This research was supported by the IBM T. J. Watson Research Center and by the National Science Foundation, Grant ECS-83-10213.

References

CROWDER, H., E. L. JOHNSON AND M. W. PADBERG. 1983. Solving Large-Scale Zero-One Linear Programming Problems. *Opns. Res.* **31,** 803–834.

LIN, S., AND B. W. KERNIGHAN. 1973. An Effective Heuristic Algorithm for the Travelling Salesman Problem. *Opns. Res.* **21,** 498–516.

STECKE, K. 1983. Formulations and Solution of Nonlinear Integer Planning Problems for FMS. *Mgmt. Sci.* **29,** 273–288.

This content downloaded from 169.229.32.137 on Thu, 8 May 2014 20:46:32 PM
All use subject to JSTOR Terms and Conditions

STECKE, K., AND J. BROWNE. 1985. Variations in Flexible Manufacturing Systems According to the Relevant Types of Automated Material Handling. *Material Flow* 179–185.

STECKE, K., AND F. TALBOT. 1983. Heuristic Loading Algorithms for Flexible Manufacturing Systems. In *Proceedings of the Seventh International Conference on Production Research*, Windsor, Ontario.

TANG, C. S., AND E. V. DENARDO. 1988. Models Arising from a Flexible Manufacturing Machine, Part II: Minimization of the Number of Switching Instants. *Opns. Res.* **36,** 778–784.

This content downloaded from 169.229.32.137 on Thu, 8 May 2014 20:46:32 PM
All use subject to JSTOR Terms and Conditions